ISE 331: Fundamentals of Computer Security

Practice Problems for Lectures 21 to 23 (Software Security)

Directions: Below is a set of exercises and problems you should do for practice. Answering this set of questions should be just one part of your preparation for quizzes and exams. In addition to solving these problems you should:

- Review the lecture notes.
- Review your homework assignments.
- Form a study group and test each other on the material.

Note: The solution is provided on the last page. It is strongly recommended that you work on the questions first before going to the solution.

Part I. Multiple Choice

1. Which option best describes the fundamental goal of Return-Oriented Programming (ROP)?
   A. Inject machine codes into a writable but non-executable memory region and execute them.
   B. Bypass non-executable memory by chaining together existing code that end with a return.
   C. Overwrite global variables to change program behavior without hijacking control flow.
   D. Brute-force stack overflow protection until the program crashes in a controlled way.

2. What is a Return-Oriented Programming (ROP) gadget?
   A. A complete function that the attacker calls directly.
   B. A specific buffer that the attacker overflows.
   C. A sequence of instructions ending in a ret instruction.
   D. A specialized tool used to find vulnerabilities in source code.

3. What is the primary motivation for using Return-Oriented Programming (ROP) instead of traditional shellcode injection?
   A. ROP does not require buffer overflow to achieve successful attack.
   B. ROP is not affected by Address Space Layout Randomization (ASLR).
   C. Shellcode will not run properly if it contains one or more zero byte.
   D. Shellcode cannot be executed if the stack and heap are non-executable.

4. Which is the most appropriate reason to say that Return-Oriented Programming (ROP) is strange or weird?
   A. Because ROP is programmed using existing instructions, which is 'weird'.
   B. Because ROP gadgets are often found in 'weird' or poorly written code.
   C. Because ROP only works on outdated, 'weird' computer architectures.
   D. Because the results of a ROP attack are unpredictable and 'weird'.

5. A heap buffer overflow can overwrite adjacent heap chunks. Which of the following could an attacker directly corrupt?
   A. The program counter register.
   B. The size field of the next chunk.
   C. The read-only segment of the executable.
   D. The heap's page tables.

6. Which of the following is an example of heap metadata?
   A. The contents of a user-allocated string.
   B. The stack canary value.
   C. The chunk size field.
   D. The instruction pointer.

7. A memory leak occurs when:
    A. A variable is overwritten with a pointer to sensitive data.
    B. A part of memory is allocated but is never freed.
    C. A buffer overflow writes past the end of a heap chunk.
    D. The same memory is freed twice in a row.

8. Which of the following best describes the purpose of reverse engineering?
    A. Remove comments, variable names, and type information from source code before distribution.
    B. Recover high-level design information and program logic that was lost during compilation.
    C. Generate the original source code, including variable names and comments from a compiled binary.
    D. Apply compiler optimizations that increase execution speed/efficiency, and reduce file size.

9. During compilation, many high-level details are lost. Which of the following is least likely to exist after compilation?
    A. String constants.
    B. Contents of an array.
    C. Local variable names and data types.
    D. Read-only global data in the .rodata section.

10. Which of the following best explains why traditional software license checkers are inherently vulnerable to cracking via reverse engineering?
    A. The license verification algorithm is in the binary and can be analyzed.
    B. The license verification algorithm is weak and can be brute forced.
    C. The key is already stored in the binary so it can be found.
    D. There is only a limited number of keys that can be used.

Part II. True or False

1. A Return-Oriented Programming (ROP) attack involves injecting new machine code instructions into the target process's memory.

2. Control Flow Integrity (CFI) is a defense that ensures indirect branches only jump to legitimate, predefined targets.

3. Return-Oriented Programming (ROP) gadgets can only use complete functions as they were originally written by the programmer.

4. The "Wilderness" chunk represents a region of memory that has been corrupted by a stack overflow and is no longer usable by the heap allocator.

5. Calling free() on a pointer immediately returns that memory to the Operating System, reducing the total memory footprint of the process.

6. You can repeatedly free the same memory multiple times with no issues.

7. Code obfuscation techniques such as opaque predicates can mislead an analyst by introducing conditional branches where one branch is never actually executed at runtime.

8. Static reverse engineering requires running the target program in a controlled environment to observe its behavior.

9. The only effective way to forge software license keys is to observe and analyze patterns of known keys.

Part III. Short Answer

1. In a ROP chain, what is the specific role of the ret instruction at the end of each gadget?

2. Explain how does the attacker's "instruction set" in ROP differ from standard assembly programming.

3. Intel's Control-flow Enforcement Technology (CET) requires indirect jumps (including ret, jmp rax, call rdx, etc) MUST end up at an endbr64 instruction or the program will terminate. Explain how does this significantly complicate ROP exploitation.

4. Explain the difference between a stack-allocated variable and a heap-allocated variable in terms of security risk. Why does heap allocation introduce the possibility of use-after-free and memory leaks?

5. Explain how overwriting a heap chunk's "size" field could lead to an "overlapping allocation." Why is this state desirable for an attacker?

6. A student copied another student's C source code when submitting homework. To avoid detection, the student changed all variable names in the source code (a becomes num1, b becomes num2, etc.). Can this avoid detection of a plagiarism detection system? Assume that the system analyzes compiled binaries from all students to detect plagiarism.

7. Compare static and dynamic reverse engineering approaches. Describe a scenario where dynamic analysis can achieve what static reverse engineering can't.

Part I Solution:
1-5 BCDAB 6-10 CBBCA

Part II Solution:
1-5 FTFFF 6-9 FTFF

Part III Solution:

1. It pops the next address from the stack into the instruction pointer (RIP/EIP), effectively "linking" to the next gadget in the chain. This connects all the ROP gadgets together to form a functional program.

2. Standard programming uses a fixed instruction set (like x86-64) to perform intended tasks. In ROP, the attacker's "instruction set" is whatever gadgets that can be found in the binary. The attacker does not write instructions but arranges addresses on the stack to force the CPU to execute instructions in an unintended sequence.

3. Most gadgets that the attacker use without CET do not begin with endbr64. With CET, the attackers can only use a drastically reduced number of gadgets, making traditional ROP chaining much harder.

4. Stack variables are automatically removed when the function returns. However, heap variables exist until manually freed. This means a programmer may free memory while a pointer to it still exists (dangling pointer -> use after free) or may forget to free it entirely (memory leak). These mistakes can be exploited by an attacker.

5. By overwriting the size field of a chunk, an attacker can trick the allocator into thinking a chunk is larger than it actually is. This leads to "overlapping allocations," where two different pointers point to the same memory. This is desirable because the attacker can use one pointer to overwrite security-critical data held by the other pointer.

6. No. All variable names are lost during compilation, so the copycat students' compiled binary will look the same with the original, and can be detected.

7. Static analysis inspects code without running, and cannot observe runtime behavior or decode dynamically generated/self-modifying code. Dynamic analysis executes the program and monitors system calls, memory, and control flow.

   Dynamic analysis is necessary when the binary encrypts code that is only decrypted at runtime; also for capturing network traffic or specific algorithm outputs that depend on runtime inputs.